

TECHNICAL DOCUMENTATION

TITLE

Lightweight SNMP Agent for Autoca Embedded System

DETAILS

- **Name:** Mahan Hithalamakki Sundaresh
- **Role** Intern (Airport Engineering)
- **Company:** BEUMER Group
- **Internship Duration:** 6 Months (January 2026 – June 2026)
- **University:** OTH Regensburg
- **Location** Beckum, Germany
- **Date:** 26-06-2026

1. ABSTRACT

This project implements a lightweight SNMPv1 agent for the BEUMER Autoca embedded controller operating under Paradigm RTOS, which is constrained by a 2 KB thread stack and the absence of dynamic memory allocation. The agent responds to SNMP GET requests on UDP port 161, providing system identification data (*sysDescr*, *sysName*, *sysLocation*), real-time uptime (*sysUpTime* as TimeTicks), and a heartbeat counter that increments every two seconds. The complete protocol stack—including BER encoding and decoding, OID storage, UDP communication handling, and dual-thread coordination—was developed from scratch in C, comprising approximately 1,070 lines of code across seven files without the use of external libraries. The implementation was validated using Paessler PRTG Network Monitor through 2 hours of continuous polling with zero failures.

2. INTRODUCTION

2.1 Background

The BEUMER Autoca is a self-driving transport vehicle deployed in airport baggage handling systems. Multiple units operate simultaneously on guided track networks, each containing an embedded controller running Paradigm RTOS for motor control, route navigation, and communication with the central system.

2.2 Problem

Prior to this project, there was no standardized method to remotely verify whether an individual Autoca controller's software was functioning correctly. Determining device health required either physical inspection or proprietary diagnostic connections. This created significant blind spots in the monitoring infrastructure, where a controller could freeze or crash without immediate detection.

2.3 Solution Approach

SNMP (Simple Network Management Protocol) was selected for the following reasons:

- Native support in PRTG, eliminating the need for custom sensors
- UDP-based communication, making it lightweight and connectionless
- Read-only GET requests are sufficient for monitoring system health

3. OBJECTIVE

The project objectives were defined in collaboration with the BEUMER Autover software team to address the operational monitoring gap.

Primary Objectives

1. **Implement a functional SNMPv1 agent**
Ensure full protocol compliance for SNMPv1 GET requests, including correct BER encoding, community string validation, request ID echoing, and proper error handling (*noSuchName*) for unknown OIDs.
2. **Enable UDP communication on port 161**
Utilize the standard SNMP port to allow seamless integration with existing monitoring infrastructure without requiring additional firewall modifications or configuration changes.
3. **Design a static MIB structure**
Develop a fixed-capacity OID-value table (32 entries) that operates without dynamic memory allocation and ensures safe access within a dual-threaded environment.
4. **Support real-time monitoring parameters**
Provide essential system monitoring data, including system identification (*sysDescr*, *sysName*, *sysLocation*), real-time uptime (*sysUpTime* as TimeTicks), and a continuously incrementing heartbeat counter to confirm active software execution.
5. **Ensure minimal resource impact**
Guarantee that the SNMP agent does not interfere with the real-time vehicle control system, targeting less than 1% CPU usage and a total memory footprint below 16 KB.

Secondary Objectives:

6. Platform portability — Code must compile and run on both Paradigm RTOS (production) and Windows (development/testing) using compile-time platform detection.
7. Clean shutdown — Both agent threads must exit gracefully when the vehicle software shuts down, without resource leaks or data corruption.
8. Integration with PRTG — The specific monitoring tool used at the Beckum installation must successfully discover and poll the agent without custom sensor configuration.

4. SYSTEM ARCHITECTURE

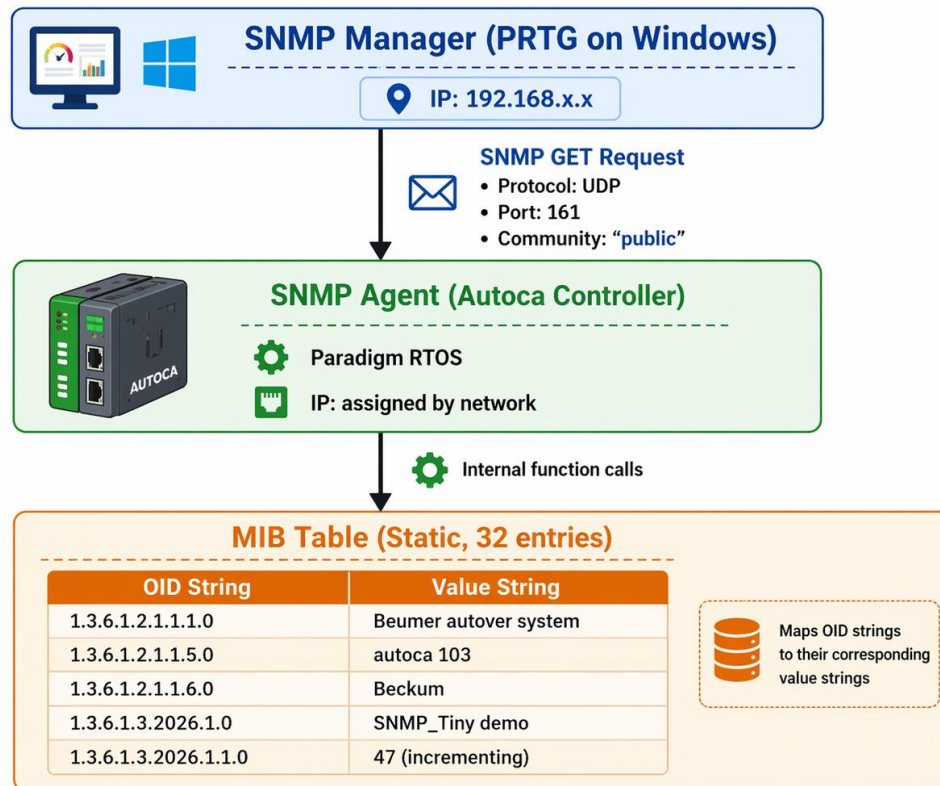
4.1 Overview

The system follows the standard SNMP Manager–Agent architecture. In this setup, the Autoca embedded controller operates as the SNMP agent, responsible for responding to incoming requests. An external monitoring system, implemented using Paessler PRTG Network Monitor running on a PC, functions as the SNMP manager, initiating queries to retrieve system information.

Communication between the manager and agent is performed over UDP using port 161, which is the standard port assigned for SNMP operations. The manager periodically sends SNMP GET requests to the agent, which processes these requests, retrieves the corresponding data from its internal MIB structure, and returns appropriately encoded SNMP responses.

This architecture enables centralized and standardized monitoring of distributed Autoca controllers, aligning embedded system diagnostics with widely used network management practices.

Communication is strictly unidirectional in initiation: the Manager always sends first, the Agent only responds. There are no TRAP (unsolicited notification) messages in this implementation.



4.2 Components

The system consists of three primary components: the SNMP Manager, the SNMP Agent, and the Management Information Base (MIB).

SNMP MANAGER (EXTERNAL)

- Paessler PRTG Network Monitor running on a Windows PC
- Configured with standard SNMP sensor pointing at the Autoca IP address
- Polls at configurable intervals (validated at 10-second intervals)
- Displays response values on dashboard, alerts on timeout (device unreachable)

SNMP AGENT (AUTOCA CONTROLLER)

- Embedded software module integrated into the existing Autoca 3.0 codebase
- Runs as two additional RTOS threads alongside the vehicle control software
- Initialized during boot (MyApp_InitSNMP()), shut down during halt (_MyApp())

- Does not share any data structures with the vehicle control code except the car number (read-only via `GloStore_GetAutoNbr()`)

MIB (MANAGEMENT INFORMATION BASE)

- Internal static array of 32 `SNMP_MIB_ENTRY` structures
- Each entry: 128-byte OID string + 256-byte value string + type tag
- Populated at boot with system identification OIDs
- Updated every 2 seconds by the heartbeat thread (tick counter only)
- Read by the UDP listener thread on each incoming GET request

5. SOFTWARE MODULES

5.1 `snmp_mib.c` — Data Storage Layer

Purpose:

This module provides the OID-to-value database used by the SNMP agent to respond to incoming SNMP GET requests.

Design Rationale:

A simple flat array structure was selected over more complex data structures such as trees or hash tables for the following reasons:

- Only a small number of OIDs are currently registered (6 OIDs), while the allocated capacity of 32 entries allows for future expansion
- Linear search across 32 entries completes within microseconds on the target CPU, ensuring acceptable performance
- Avoids the use of dynamic memory allocation (`malloc`), eliminating risks related to fragmentation and memory leaks
- Requires no pointer-based data management, simplifying implementation and improving reliability
- The entire MIB is stored in a single contiguous memory block, ensuring predictable memory layout and efficient access

Data Structure

```
#define MAX_OIDS 32
```

```

typedef struct _SNMP_MIB_ENTRY
{
    char      oid[128];  /* "1.3.6.1.2.1.1.1.0" */
    char      value[256]; /* "Beumer autover system" */
    SNMP_MIB_TYPE type;    /* STRING or TIMETICKS */
    BOOL       bDynamic;  /* reserved for future use */
} SNMP_MIB_ENTRY;

```

```

static SNMP_MIB_ENTRY g_Mib[MAX_OIDS];
static INT          g_Count = 0;
static BOOL         g_bReady = FALSE;

```

API Functions

Safety Mechanism: The g_bReady flag acts as a gate. All public functions check this flag before accessing the array. This prevents:

- Reads before initialization (boot sequence not yet complete)
- Reads/writes after shutdown (MIB cleared, threads still draining)

Thread Safety: No mutex is used. Safety relies on the access pattern:

- Single writer: heartbeat thread calls AddOrUpdate() every 2 seconds
- Single reader: UDP thread calls Get() on each request
- Writes are memcpy into fixed buffers (atomic enough on this single-core CPU)
- No entries are ever deleted, only added or overwritten

Function	Parameters	Behavior

SNMP_MIB_Init()	None	Resets the entry count to 0 and sets g_bready to TRUE
SNMP_MIB_Done()	None	Resets the entry count to 0 and sets g_bReady to FALSE
SNMP_MIB_AddOrUpdate(oid, value)	OID string, value string	Searches for an existing OID; overwrites the value if found, otherwise creates a new entry. Returns FALSE if the table is full
SNMP_MIB_Get(oid, out, len)	OID string, output buffer, buffer size	Performs a linear search and copies the value if found into the output buffer. Returns TRUE or FALSE

5.2 SNMP_UDP.C — NETWORK COMMUNICATION LAYER

Purpose

This module implements the UDP listener thread responsible for receiving SNMP GET requests from the network, coordinating parsing and lookup operations through other modules, and sending responses back to the requesting manager. It serves as the central orchestrator of the SNMP request–response cycle.

Design Rationale

A dedicated listener thread was selected instead of a polling-based approach for the following reasons:

- The RTOS provides blocking socket receive with timeout (`ApiSock_recvfrom`)
- Blocking for 2 seconds with periodic stop-flag checks enables responsive shutdown without busy-waiting
- A separate thread ensures that SNMP processing does not interfere with the vehicle control loop
- The orchestrator design cleanly separates responsibilities, keeping protocol logic (`snmp_v1.c`) and data management (`snmp_mib.c`) independent of network I/O

Thread Configuration

Property	value
Thread Name	"SNMP" (limited to 4 characters by RTOS)
Stack Size	2048 bytes
Priority	30 (low priority, below vehicle control tasks)
Entry Point	<code>SNMP_UDP_Thread_Wrapper()</code> → <code>SNMP_UDP_Thread()</code>
Receive Timeout	2000 ms per cycle

Static Buffers

Due to the 2 KB thread stack limitation, all buffers are statically allocated:

- Receive buffer for incoming UDP packets
- Transmit buffer for SNMP responses
- Temporary parsing buffers for OID extraction and message handling

This design ensures predictable memory usage and prevents stack overflow in the constrained RTOS environment.

Main Loop Logic:

```
while (!SNMP_Tiny_GetStopflag())
{
    /* Receive incoming SNMP packet (blocking with 2s timeout) */
    recvLen = ApiSock_recvfrom(&sock, recvBuf, 1024, 2000ms (about 4 seconds), &fromIP,
&fromPort);

    /* Timeout or no data → check stop flag again */
    if (recvLen <= 0)
        continue;

    /* Extract requested OID from SNMP packet */
    if (!SNMPv1_ExtractRequestedOid(...))
        continue; /* Invalid or malformed packet */

    /* Process request based on OID */
    if (OID == sysUpTime)
    {
        /* Build dynamic TimeTicks response */
        BuildTimeTicksResponse(...);
    }
    else if (SNMP_MIB_Get(...) == TRUE)
    {
        /* Build string response from MIB */
        BuildStringResponse(...);
    }
    else
    {
```

```

    /* Unknown OID → return SNMP error */
    BuildErrorResponse(...); /* noSuchName */
}

/* Send response back to requesting manager */
ApiSock_sendto(&sock, sendBuf, outLen, fromIP, fromPort);
}

```

sysUpTime Special Handling

The OID **1.3.6.1.2.1.1.3.0** (*sysUpTime*) is handled outside the standard MIB table for the following reasons:

1. **Correct SNMP Data Type Requirement**

SNMP managers expect *sysUpTime* to be encoded using BER tag **0x43 (TimeTicks)** rather than **0x04 (OCTET STRING)**. Returning the value as a string leads to rejection by monitoring systems such as PRTG.

2. **Real-Time Value Accuracy**

The uptime value must reflect the exact moment when the request is processed. Storing it in the MIB would introduce latency (up to 2 seconds due to update intervals), resulting in stale or inaccurate data.

3. **Efficient Runtime Calculation**

The uptime is computed dynamically using the system tick counter:

API Functions

Function	Visibility	Purpose
SNMP_UDP_Thread()	Public (declared in snmp_tiny.h)	Main listener loop that handles SNMP request-response processing. Returns 0 on clean exit and 1 on socket failure
SNMP_UDP_Thread_Wrapper()	Public	RTOS thread entry point that invokes SNMP_UDP_Thread() and performs cleanup via ApiThrd_Cleanup()
StartSnmpUdpListener()	Public (legacy stub)	No operational functionality. Retained for backward compatibility with older code that previously invoked this function

Error Handling:

Condition	Behaviour
ApiSock_Open fails	Print error, return 1 (thread exits)
ApiSock_Bind fails	Close socket, print error, return 1
Receive timeout (recvLen <= 0)	Continue loop (check stop flag)
Invalid SNMP packet	Silently ignored (continue)
OID found in MIB	Send string response
OID is sysUpTime	Send TimeTicks response
OID not found	Send noSuchName error response
Output buffer overflow	BuildErrorResponse returns 0, no send

Debug Output

Each successful SNMP response is logged to the console to provide visibility during development and debugging:

```
printf("[SNMP] OID %s -> \"%s\"\n", reqOid, value);    /* string */
printf("[SNMP] OID %s -> TimeTicks %lu\n", reqOid, ulUptime); /* uptime */
printf("[SNMP] OID %s -> noSuchName\n", reqOid);      /* error */
```

These debug statements are retained in the production build because the Paradigm RTOS routes printf output to a dedicated debug serial port. This port is only active when a diagnostic cable is connected, ensuring that the logging mechanism does not impact performance during normal system operation.

Shutdown Behavior

When `SNMP_Tiny_GetStopflag()` returns TRUE, the UDP listener thread performs a controlled shutdown sequence:

1. The main processing loop exits
2. The socket is closed using:

```
ApiSock_Close(&sock);
```

A confirmation message is printed:

```
printf("[SNMP] UDP listener stopped.\n");
```

- The function returns 0, indicating a clean shutdown
- `SNMP_UDP_Thread_Wrapper()` subsequently calls `ApiThrd_Cleanup()` to release the thread handle

Shutdown Timing Guarantee

The configured 2-second receive timeout ensures that the thread periodically checks the stop flag. As a result, the listener thread is guaranteed to terminate within a maximum of 2 seconds after a shutdown request is issued.

5.3 SNMP_V1.C — PROTOCOL PROCESSING LAYER

Purpose

This module implements the BER (Basic Encoding Rules) encoding and decoding engine responsible for SNMP protocol processing. It parses raw SNMP packet bytes into usable data fields and constructs valid SNMP response packets. With over 550 lines of code, this is the most complex module in the system and was implemented entirely without external ASN.1 or SNMP libraries.

Design Rationale

A custom, from-scratch implementation was required due to the following constraints:

- No SNMP libraries are available for the Paradigm RTOS platform
- Existing libraries such as Net-SNMP depend on POSIX features, dynamic memory allocation, and a full C runtime environment, none of which are available
- The system only requires support for SNMPv1 GET requests with a fixed community string ("public"), making full-stack libraries unnecessary
- A custom implementation provides precise control over buffer sizes, memory layout, and stack usage

Responsibilities

1. Packet Validation

Verifies the structure and correctness of incoming SNMP packets:

- a. SEQUENCE tag presence
- b. SNMP version (must be 0 for SNMPv1)
- c. Community string (must be "public")
- d. PDU type (must be GET-REQUEST, tag 0xA0)

2. OID Extraction

Decodes BER-encoded OID fields into human-readable dotted string format (e.g., 1.3.6.1.2.1.1.1.0).

3. Response Construction

Builds complete BER-encoded GET-RESPONSE packets containing:

- a. Correct value types (OCTET STRING or TimeTicks)
- b. Properly formatted SNMP message structure

4. Error Handling

Generates valid SNMP error responses (*noSuchName*) when requested OIDs are not found in the MIB.

Public API

Function	Called By	Purpose
SNMPv1_ExtractRequestedOid()	snmp_udp.c	Parses incoming packet and extracts the requested OID as a dotted string
SNMPv1_BuildGetResponse_Public_String()	snmp_udp.c	Builds a response containing an OCTET STRING value
SNMPv1_BuildGetResponse_Public_TimeTicks()	snmp_udp.c	Builds a response containing a TimeTicks value
SNMPv1_BuildErrorResponse_Public()	snmp_udp.c	Constructs a noSuchName error response

Internal Helper Functions (static)

BER READERS

- ber_get_len
- ber_get_integer
- ber_get_octets
- ber_get_oid_val_ptr

BER WRITERS

- ber_put_len
- ber_put_int
- ber_put_octets
- ber_put_timeticks
- ber_put_oid

OID UTILITIES

- ber_oid_equals
- ber_decode_oid_to_string

STRING HELPERS

- SNMP_UlongToStr

5.4.1 BER ENCODING OVERVIEW

BER (Basic Encoding Rules, ITU-T X.690) defines the binary wire format used for all SNMP communication. Each field follows a **Tag–Length–Value (TLV)** structure:

[Tag: 1 byte] [Length: 1–3 bytes] [Value: N bytes]

These elements are hierarchically nested to form a complete SNMP message.

SNMP GET Request Structure

SEQUENCE (0x30)	← outer container
INTEGER (0x02) = 0	← version (SNMPv1)
OCTET STRING (0x04) = "public"	← community
GET-REQUEST (0xA0)	← PDU type
INTEGER (0x02) = 12345	← request-id
INTEGER (0x02) = 0	← error-status
INTEGER (0x02) = 0	← error-index
SEQUENCE (0x30)	← VarBindList
SEQUENCE (0x30)	← VarBind
OID (0x06) = 1.3.6.1.2.1.1.1.0	← requested OID
NULL (0x05)	← placeholder

SNMP GET Response Differences

- PDU tag changes from **0xA0 (GET-REQUEST)** to **0xA2 (GET-RESPONSE)**
- The NULL field is replaced with the actual value:
 - **OCTET STRING** (for text values)
 - **TimeTicks** (for uptime values)

5.4.2 BER Tag Values Used

Tag	Hex	Constant
SEQUENCE	0x30	TAG_SEQUENCE
INTEGER	0x02	TAG_INTEGER

OCTET STRING	0x04	TAG_OCTET
NULL	0x05	TAG_NULL
OID	0x06	TAG_OID
GET-REQUEST	0xA0	PDU_GET
GET-RESPONSE	0xA2	PDU_GETRESP
TimeTicks	0xA3	TAG_TIMETICKS

5.4.3 BER LENGTH ENCODING

BER supports two encoding formats for length:

Form	Condition	Encoding
Short	Length < 128	Single byte
Long	Length ≥ 128	0x82 + 2-byte big-endian length

Design Choice

- Parsing supports both **short and long forms**
- Response generation uses **only long form** for consistency and simplicity

Length Patching Strategy

Since the total length of nested structures is unknown beforehand, the implementation uses placeholder bytes:

```
/* Step 1: Write tag and reserve space */
```

```
*q++ = TAG_SEQUENCE;
```

```
msgLenPos = q;
```

```
q += BER_LEN_RESERVE; /* reserve 3 bytes */
```

```
/* Step 2: Write nested content */
```

```
q = ber_put_int(q, version);
```

```
q = ber_put_octets(q, community, 6);
```

```
/* ... additional fields ... */
```

```
/* Step 3: Patch actual length */
```

```
ber_put_len(msgLenPos, (UINT)(q - (msgLenPos + BER_LEN_RESERVE)));
```

Nesting Levels

Length patching is applied at four levels:

- Message
- PDU
- VarBindList
- VarBind

Patching is performed **from inner to outer structures**.

5.4.4 OID ENCODING RULES

OID encoding follows rules defined in X.690.

Rule 1: First Two Sub-Identifiers

The first two values are combined:

$\text{encoded_byte} = \text{first_subid} \times 40 + \text{second_subid}$

Example:

$1.3 \rightarrow (1 \times 40 + 3) = 43 = 0x2B$

Rule 2: Remaining Sub-Identifiers

- Values < 128 → encoded as **single byte**
- Values ≥ 128 → encoded using **base-128 multi-byte format**
 - High bit (0x80) set for all bytes except the last

Example OID Encoding

OID:

1.3.6.1.2.1.1.1.0

Encoded bytes:

2B	(1*40 + 3)
06	(6)
01	(1)
02	(2)
01	(1)
01	(1)
01	(1)
00	(0)

5.4.5 BASE-128 ENCODING EXAMPLE

Consider encoding the sub-identifier value **2026**:

2026 (decimal) = 11111101010 (binary)
--

Step 1: Split into 7-bit Groups

0001111 1101010

Step 2: Apply Base-128 Encoding

- All bytes except the last have the high bit (0x80) set
- The last byte has the high bit cleared

Encoded bytes: 0x8F 0x6A

- 0x8F → high bit set (continuation)
- 0x6A → last byte (termination)

5.4.6 OID DECODING (BER TO STRING)

The function `ber_decode_oid_to_string()` converts BER-encoded OID bytes into a human-readable dotted string.

Decoding Algorithm

1. **First byte:**
 - a. Divide by 40 → first sub-identifier
 - b. Modulo 40 → second sub-identifier
2. **Remaining bytes:**
 - a. Accumulate base-128 chunks
 - b. Continue until a byte without the 0x80 bit is encountered
3. **Output format:**
 - a. Convert each sub-ID to a decimal string
 - b. Concatenate using dot (.) separators

Example

```
/* Input bytes: [2B 06 01 02 01 01 01 00]
*
* 0x2B = 43 → 43/40 = 1, 43%40 = 3 → "1.3"
* 0x06 = 6 → ".6"
* 0x01 = 1 → ".1"
* ...
* Result: "1.3.6.1.2.1.1.1.0"
*/
```

The helper function `SNMP_UlongToStr()` is used to convert numeric sub-identifiers into strings, as `sprintf` is unavailable in the Paradigm RTOS environment.

5.4.7 REQUEST PARSING FLOW

Each `Build*` function processes incoming SNMP packets using the following validation sequence:

1. Check outer SEQUENCE tag (0x30)
2. Read message length
3. Read INTEGER → must be 0 (SNMPv1)
4. Read OCTET STRING → must be "public"
5. Check PDU tag → must be 0xA0 (GET-REQUEST)
6. Read PDU length
7. Read INTEGER → request-id (store for response)
8. Read INTEGER → error-status (ignored)
9. Read INTEGER → error-index (ignored)
10. Check VarBindList SEQUENCE tag
11. Check VarBind SEQUENCE tag
12. Read OID field → raw BER bytes
13. Check NULL tag and length (must be 0)
14. Compare OID with expected value

Failure Handling:

If any validation step fails (invalid tag, incorrect length, community mismatch), the function immediately returns 0. The packet is silently dropped by `snmp_udp.c`.

5.4.8 RESPONSE BUILDING STRATEGY

After successful parsing, the response is built in a single output buffer.

Buffer Layout

[30][82 xx xx]	← SEQUENCE
[02 01 00]	← INTEGER version = 0
[04 06 "public"]	← OCTET STRING community
[A2][82 xx xx]	← GET-RESPONSE
[02 xx reqId]	← request-id (echoed)
[02 01 00]	← error-status = 0
[02 01 00]	← error-index = 0
[30][82 xx xx]	← VarBindList
[30][82 xx xx]	← VarBind
[06 xx OID bytes]	← OID
[04 xx value bytes]	← OCTET STRING value

OR
[43 xx ticks bytes] ← TimeTicks value

Length Patching Order

```
ber_put_len(vbLenPos, ...); /* 1. VarBind */
ber_put_len(vblLenPos, ...); /* 2. VarBindList */
ber_put_len(pduLenPos, ...); /* 3. PDU */
ber_put_len(msgLenPos, ...); /* 4. Message */
```

Patching is performed from the innermost structure outward.

5.4.9 ERROR RESPONSE (NOSUCHNAME)

When an OID is not present in the MIB, the response follows RFC 1157:

- error-status = 2 (noSuchName)
- error-index = 1 (first VarBind caused the error)
- VarBindList is copied unchanged from the request

```
q = ber_put_int(q, reqId);
q = ber_put_int(q, SNMP_ERR_NOSUCHNAME);
q = ber_put_int(q, SNMP_ERR_INDEX_1);
```

```
/* Copy original VarBindList */
memcpy(q, vblStart, vblLen);
q += vblLen;
```

This avoids re-encoding the OID, improving efficiency.

5.4.10 TIMETICKS ENCODING EDGE CASE

SNMP TimeTicks is an **unsigned 32-bit value**, but BER INTEGER encoding is **signed**.

If the most significant bit is set (> 2,147,483,647), a leading 0x00 byte must be added:

```
if (buf[0] & 0x80)
{
    *p++ = 5; /* length */
    *p++ = 0x00; /* sign padding */
}
```

```

    *p++ = buf[0]; *p++ = buf[1];
    *p++ = buf[2]; *p++ = buf[3];
}

```

5.4.11 OID COMPARISON STRATEGY

Instead of decoding received OIDs into strings, the implementation encodes the expected string and performs a byte comparison:

```

static int ber_oid_equals(const UCHAR* oidPtr, UINT oidLen, const CHAR* expected)
{
    UCHAR tmp[128];
    UCHAR* t = ber_put_oid(tmp, expected);
    UINT encLen = (UINT)(t - tmp - 2);

    if (encLen != oidLen)
        return 0;

    return memcmp(oidPtr, tmp + 2, oidLen) == 0;
}

```

Advantages

- Faster than string conversion
- No dynamic memory required
- Direct byte-level comparison

5.4.12 Validation Summary

Check	Failure Condition	Result
First byte != 0x30	Not a valid SEQUENCE	Packet ignored

Version != 0	Not SNMPv1	Packet ignored
Community != "public"	Unauthorized access	Packet ignored
PDU tag != 0xA0	Not GET-REQUEST	Packet ignored
OID mismatch	Unexpected OID	Skipped
NULL tag missing	Malformed packet	Packet ignored
Length overflow	Corrupt/truncated packet	Packet ignored

Safety Guarantee

All length values are bounds-checked before memory access.

This ensures:

- No buffer overflow
- No out-of-bounds reads
- Robust handling of malformed packets

5.5 SNMP_TINY.C — SUBSYSTEM LIFECYCLE AND DATA GENERATION LAYER

Purpose

This module manages the complete SNMP subsystem lifecycle, including initialization, shutdown, and coordinated thread termination. It also runs a background heartbeat thread that periodically updates the MIB with an incrementing counter every 2 seconds, providing continuous confirmation that the controller software is operational.

Design Rationale

Separating lifecycle management from network communication and protocol processing offers the following advantages:

- Provides a single entry point (SNMP_Tiny_Init) for the main application to initialize the SNMP subsystem
- Provides a single exit point (SNMP_Tiny_Done) for controlled and predictable shutdown
- Uses a shared context structure that enables both threads to coordinate termination through a common stop flag
- Enables heartbeat generation independently of network activity, ensuring system liveness is continuously indicated even when no SNMP manager is polling

Thread Configuration

Property	Value
Thread name	"SNTY" (limited to 4 characters by RTOS)
Stack size	2048 bytes
Priority	30 (low priority, below vehicle control tasks)
Cycle time	2000 ms (sleep interval between updates)
Entry point	SNMP_Tiny_Thread_Wrapper() → SNMP_Tiny_Thread()

OIDs Managed by This Module

```
static const CHAR* OID_DEMO    = "1.3.6.1.3.2026.1.0";
```

```
static const CHAR* OID_TICK    = "1.3.6.1.3.2026.1.1.0";
```

```
static const CHAR* OID_SYS_DESCR = "1.3.6.1.2.1.1.1.0";
```

```
static const CHAR* OID_SYS_NAME   = "1.3.6.1.2.1.1.5.0";
static const CHAR* OID_SYS_LOCATION = "1.3.6.1.2.1.1.6.0";
```

Boot Sequence

Called from MyApp_InitSNMP() during system startup:

SNMP_Tiny_Init():

1. memset(&gSnmptCtx, 0, sizeof(gSnmptCtx)) → clear context
2. SNMP_MIB_Init() → table ready (g_bReady = TRUE)
3. SNMP_MIB_AddOrUpdate(OID_DEMO, "SNMP_Tiny demo") → static test value
4. SNMP_Tiny_RegisterSystemOIDs() → sysDescr, sysName, sysLocation
5. ApiThrd_Create("SNTY", SNMP_Tiny_Thread_Wrapper, ...) → start heartbeat
6. Return TRUE

System OID Registration

SNMP_Tiny_RegisterSystemOIDs() populates the MIB with standard MIB-II system group OIDs at boot:

OID	Standard Name	Value
1.3.6.1.2.1.1.1.0	sysDescr	"Beumer autover system"
1.3.6.1.2.1.1.5.0	sysName	"autoca 103"
1.3.6.1.2.1.1.6.0	sysLocation	"Beckum"

Dynamic sysName construction:

The car number is read from the global store at runtime and appended to the prefix "autoca ":

```
CHAR szSysName[64];
```

```
INT pos = 0;
```

```
SNMP_SafeStrAppend(szSysName, sizeof(szSysName), &pos, "autoca ");
```

```
UINT autoNbr = GloStore_GetAutoNbr();
```

```
CHAR numBuf[12];
```

```
SNMP_UintToStr(autoNbr, numBuf, sizeof(numBuf));
```

```
SNMP_SafeStrAppend(szSysName, sizeof(szSysName), &pos, numBuf);
```

```
SNMP_MIB_AddOrUpdate(OID_SYS_NAME, szSysName);
```

This produces "autoca 1", "autoca 103", etc. depending on the vehicle. The SNMP_SafeStrAppend helper is used instead of strcat because strcat is not reliably available on Paradigm RTOS.

Heartbeat Loop

The SNTY thread runs this loop for the lifetime of the application:

```
UINT SNMP_Tiny_Thread(VOID) { UINT tick = 0; CHAR buf[32];
while (!gSnmptx.bStopFlag)
{
    ApiOS_Sleep(SNMP_TINY_CYCLE_MS);    /* 2000 MS */

    if (gSnmptx.bStopFlag)
        break;

    tick++;
    SNMP_UintToStr(tick, buf, sizeof(buf));
    SNMP_MIB_AddOrUpdate(OID_TICK, buf);
}

return 0;

}
```

Shutdown Sequence

Called from _MyApp() during system halt:

SNMP_Tiny_Done():

- 1. gSnmptx.bStopFlag = TRUE** → signal both threads to exit
- 2. ApiOS_Sleep(2500)** → wait for threads to see flag and exit
- 3. SNMP_MIB_Done()** → disable table (g_bReady = FALSE)

A 2500 MS wait guarantees both threads have completed at least one full loop iteration after the flag was set, ensuring they've exited cleanly before SNMP_MIB_Done() clears the table. Without this wait, a thread could be mid-way through accessing the MIB when it's cleared — causing a read of stale/zeroed memory.

API Functions

Function	Visibility	Purpose
SNMP_Tiny_Init()	Public	Full subsystem startup (MIB + OIDs + heartbeat thread)
SNMP_Tiny_Done()	Public	Full subsystem shutdown (stop flag + wait + MIB clear)
SNMP_Tiny_Thread()	Public	Heartbeat loop (called by wrapper)
SNMP_Tiny_SetStopflag()	Public	Set shared stop flag
SNMP_Tiny_GetStopflag()	Public	Read shared stop flag
SNMP_Tiny_SetUdpThread()	Public	Store UDP thread handle
SNMP_Tiny()	Public (legacy stub)	No-op, kept for link compatibility
_SNMP_Tiny()	Public (legacy stub)	No-op, kept for link compatibility

6. MULTITHREADING DESIGN

The SNMP agent implementation uses two cooperating threads that share a single context structure. This design separates network communication from data generation, ensuring reliability and minimal impact on the real-time vehicle control system.

```
typedef struct _SNMP_TINY_CTX
```

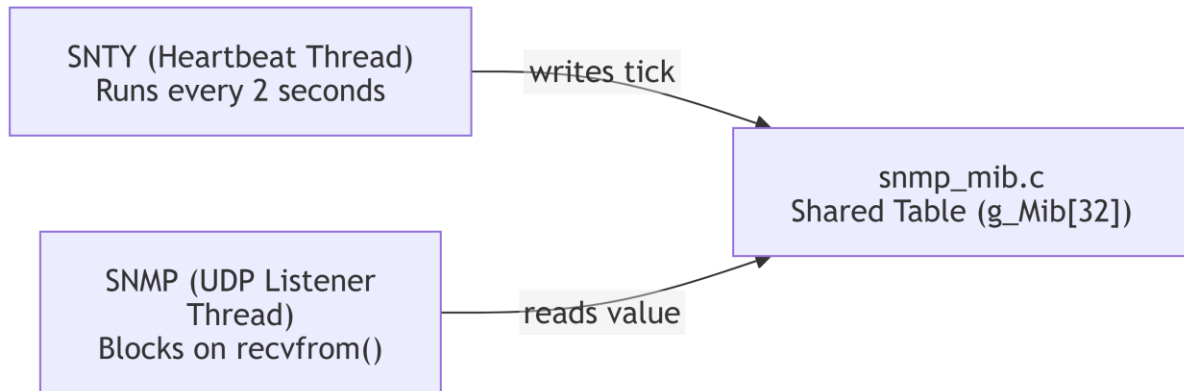
```
{
```

```
    P_API_THREAD tinyThreadPtr; /* heartbeat thread ("SNTY") */
```

```
    P_API_THREAD udpThreadPtr; /* UDP listener thread ("SNMP") */
```

```
    BOOL    bStopFlag; /* TRUE = both threads must exit */
```

```
} SNMP_TINY_CTX;
```



7. DATA FLOW (END-TO-END)

The complete SNMP request-response cycle is described below, including the responsible module and function at each step.

Request-Response Flow

Step	Action	Module	Function
1	PRTG sends SNMP GET request to Autoca (UDP port 161)	External	-----
2	UDP socket receives raw packet	snmp_udp.c	ApiSock_recvfrom()
3	Extract OID from BER-encoded packet	snmp_v1.c	SNMPv1_ExtractRequestedOid()
4	Look up OID value in MIB	snmp_mib.c	SNMP_MIB_Get()

5a	If found: build string response	snmp_v1.c	SNMPv1_BuildGetResponse_Public_String()
5b	If sysUpTime: build TimeTicks response	snmp_v1.c	SNMPv1_BuildGetResponse_Public_TimeTicks()
5c	If not found: build error response	snmp_v1.c	SNMPv1_BuildErrorResponse_Public()
6	Send BER-encoded response back to manager	snmp_udp.c	ApiSock_sendto()
7	PRTG displays the value on dashboard	External	-----

Timing Characteristics

- Processing time per request:
Less than 1 ms (dominated by MIB lookup and BER encoding)
- Response behavior:
Responses are generated immediately upon request reception
- Receive timeout (2 seconds):
Used only for periodic stop flag checks — does not affect response latency

8. EXAMPLE OIDS

OID Reference Table

OID	Standard Name	Type	Value	Update Frequency
1.3.6.1.2.1.1.1.0	sysDescr	OCTET STRING	"Beumer autover system"	Static (set at boot)
1.3.6.1.2.1.1.3.0	sysUpTime	TimeTicks	Hundredths of seconds since agent start	Calculated live per request
1.3.6.1.2.1.1.5.0	sysName	OCTET STRING	"autoca 103" (number varies per vehicle)	Static (set at boot)
1.3.6.1.2.1.1.6.0	sysLocation	OCTET STRING	"Beckum"	Static (set at boot)
1.3.6.1.3.2026.1.0	(custom) demo	OCTET STRING	"SNMP_Tiny demo"	Static (set at boot)
1.3.6.1.3.2026.1.1.0	(custom) heartbeat	OCTET STRING	"1", "2", "3", ...	Every 2 seconds

OID Tree Placement:

- 1.3.6.1.2.1.1.* — Standard MIB-II system group (RFC 1213). These OIDs are recognized by all SNMP managers without custom MIB file import.
- 1.3.6.1.3.2026.* — Experimental branch. Used for Beumer-specific custom OIDs. The number 2026 was chosen as the development year.

PRTG Sensor Configuration: No custom MIB file is needed. PRTG's standard "SNMP Library" sensor auto-discovers sysDescr, sysName, sysLocation, and sysUpTime. The heartbeat OID can be added as a custom "SNMP Custom String" sensor with the full OID entered manually.

9. Testing and Validation

The testing phase aimed to verify that the SNMP agent:

- Correctly implements the SNMPv1 protocol
- Responds accurately to SNMP GET requests
- Maintains stability under continuous operation
- Operates within platform resource constraints
- Integrates seamlessly with standard monitoring tools such as PRTG

```
C:\beumer\obj\autoWinDOS\ x + v
dT:000 TSum:031 ApiOS Start Task 0
dT:000 TSum:031 SimPic Start Task 0
dT:000 TSum:031 SimFU3 Start Task 0
dT:000 TSum:031 SimFu1 Start Task 0
dT:000 TSum:031 SimExMod_EA Start Task 0
dT:000 TSum:031 SimEA Start Task 0
dT:110 TSum:141 ConRemo Start Task... 0
dT:093 TSum:234 ConAuto Start Task 0
dT:110 TSum:344 ConBeAS Start Task... 0
dT:000 TSum:344 IAC Start Task 0
dT:000 TSum:344 DRIVE Start Task 0
dT:000 TSum:344 TASKS Start Task 0
dT:000 TSum:344 Wlan Init Converter 0
dT:000 TSum:344 Wlan Start Task 0
dT:000 TSum:344 WORK Start Task 0
dT:000 TSum:344 Try Start SNMP Tiny Agent... 0 [SNMP] Registered sysDescr=Beumer autover system sysName=autoca 1 sysLoc
ation=Beckum

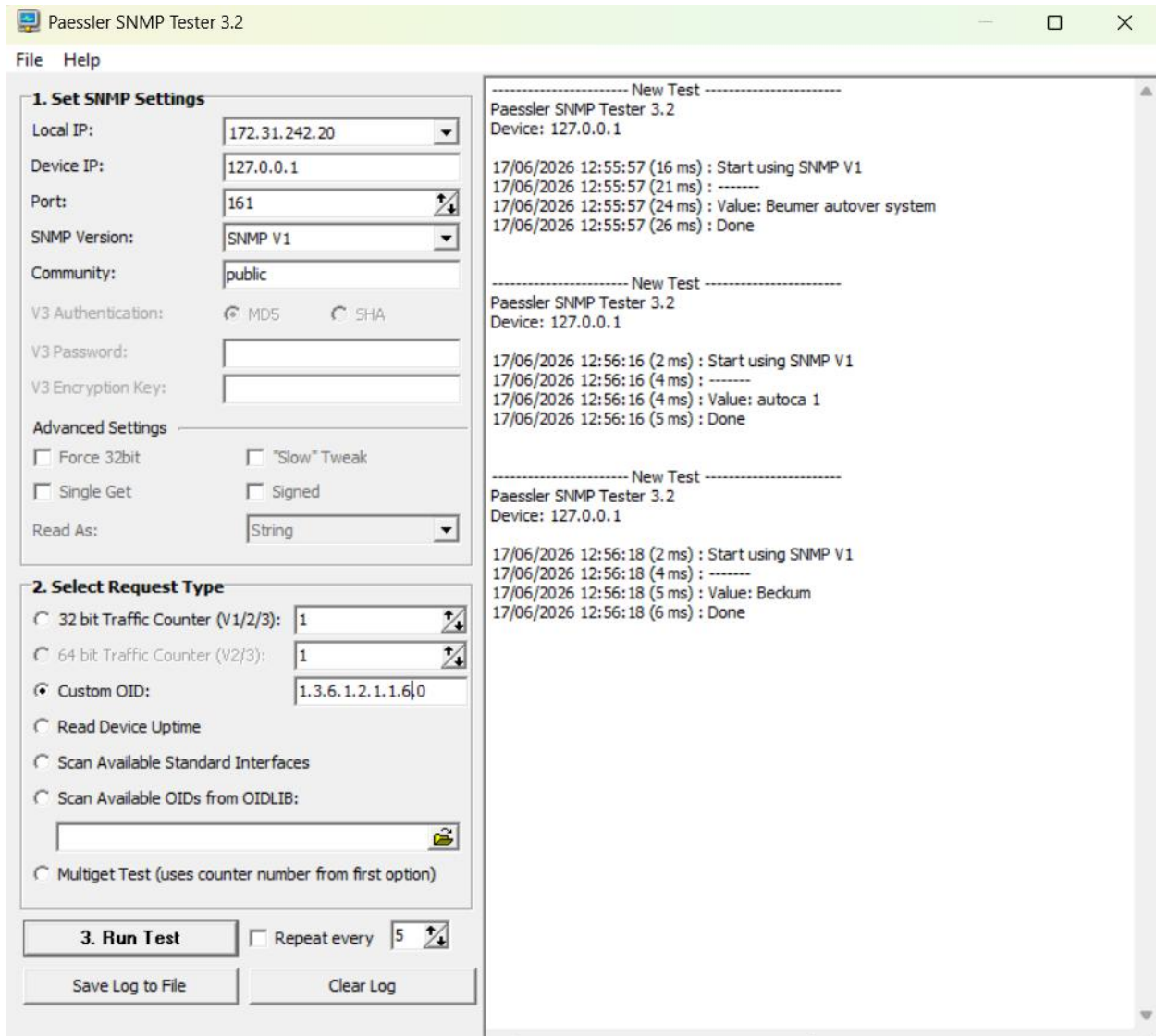
[SNMP] UDP listener started

dT:000 TSum:344 SNMP Tiny Agent started 0 The NCBRESET on LANA 5 return code is: 0x0
[SNMP] UDP listener active on port 161

dT:2000 TSum:2344 Init OK autoca l\uft... 0 [SNMP] OID 1.3.6.1.2.1.1.6.0 -> "Beckum"
[SNMP] OID 1.3.6.1.2.1.1.3.0 -> TimeTicks 509
[SNMP] OID 1.3.6.1.2.1.1.5.0 -> "autoca 1"
[SNMP] OID 1.3.6.1.2.1.1.3.0 -> TimeTicks 984
[SNMP] OID 1.3.6.1.2.1.1.1.0 -> "Beumer autover system"
[SNMP] OID 1.3.6.1.2.1.1.3.0 -> TimeTicks 1450
```

```
View Clipboard
Chiptool Terminal (10.7.253.103)
File View Connection
Task *IAC TaskId 4584
IAC Start Task 0 1563 ms [total]
Task *DRV TaskId 4676
DRIVE Start Task 0 1564 ms [total]
Task *TSK TaskId 4768
TASKS Start Task 0 1586 ms [total]
Task *BAT TaskId 4860
Wlan Init Converter 0 3300 ms [total]
Task *WLN TaskId 4952
Wlan Start Task 0 3313 ms [total]
Task *WRK TaskId 5044
WORK Start Task 0 3321 ms [total]
Try Start SNMP Tiny Agent... 0 3322 ms [total]
Task *TSN TaskId 5136
[SNMP] UDP listener active on port 161
Task *SNM TaskId 5228
[SNMP] UDP listener started
SNMP Tiny Agent started 0 3327 ms [total]
Init OK autoca l\uft... 0 5329 ms [total]
*****
* This is autoca 103 *
*****
[SNMP] Responded with value "HelloSNMP"
```

Connected via Telnet to 10.7.253.103:Telnet.



Conclusion

The testing and validation phase confirmed that the implemented SNMP agent:

- Fully complies with SNMPv1 GET request handling requirements
- Is stable under long-duration operation
- Provides accurate and real-time monitoring data
- Integrates seamlessly with industry-standard monitoring tools
- Meets all performance and resource constraints of the embedded platform

10. LEARNINGS

Throughout the course of this project, I have tried my best to design and implement a lightweight SNMPv1 agent within the constraints of the Autoca embedded system. Working in such a restricted environment, with limited memory, no dynamic allocation, and a constrained runtime, gave me a deeper appreciation of how carefully software must be designed for embedded platforms. It required me to think more about efficiency, predictability, and simplicity in every part of the implementation.

One of the key takeaways from this experience was understanding how to approach problems step by step, especially when working close to the hardware and without the convenience of high-level libraries. Implementing parts of the SNMP protocol, particularly BER encoding and decoding, helped me gain practical insight into how network protocols actually work at the byte level. Instead of relying on existing libraries, I had the opportunity to explore and implement the logic myself, which improved my understanding of both data representation and communication between systems.

I also learned the importance of structuring software in a modular way. Separating the system into different components such as the MIB storage, protocol handling, UDP communication, and lifecycle management made the overall design clearer and easier to maintain. This approach helped me understand how larger software systems are organized in industry, where different responsibilities must be clearly defined and kept independent.

Working with multithreading in this project was another valuable experience. Designing a system where two threads (UDP listener and heartbeat) operate independently but still coordinate safely using a shared context taught me how to think about concurrency in a practical way. Instead of using complex synchronization mechanisms, I learned how simple and well-defined access patterns can provide safe and efficient solutions.

In addition, integrating the system with PRTG Network Monitor helped me understand how embedded systems fit into larger industrial environments. It showed me the importance of using standard protocols like SNMP so that systems can easily communicate with widely used monitoring tools without requiring custom solutions.

Overall, this project helped me improve my problem-solving skills, my ability to work under constraints, and my understanding of how to design reliable and efficient embedded software. I have made a conscious effort throughout the project to approach each challenge carefully and do the best I can with the knowledge and resources available to me.

I would like to sincerely thank Christoph and Thomas for their continuous guidance, support, and valuable feedback throughout the entire project. Their suggestions and direction helped me stay focused and improve my work at every stage.